

DEVELOPMENT TASK BREAKDOWN

Notification Service

5-Developer Parallel Work Split — One PR per Developer

Document Type	Development Schedule / Task Breakdown
Component	Notification Service (Microservice)
Team Size	5 Developers
Version	1.0
Status	Ready for Sprint Kickoff
Date	July 16, 2026
Target Implementation Stack	Java 17, Spring Boot 3.x, Maven, PostgreSQL, Spring Data JPA, Jakarta Validation, springdoc-openapi (Swagger)

Revision History

Version	Date	Author	Description
1.0	2026-07-16	Solution Architecture	Initial 5-developer task split — Java 21 / MySQL
1.1	2026-07-16	Solution Architecture	Updated to Java 17 / PostgreSQL per Tech Lead request

Prepared for the Notification Service engineering team

Table of Contents

- 1. Overview & Design Principle 3
- 2. Developer 1 — Core Foundation 4
- 3. Developer 2 — Email Notification..... 5
- 4. Developer 3 — SMS Notification (Mock) 6
- 5. Developer 4 — In-App Notification..... 7
- 6. Developer 5 — Templates & History 8
- 7. Recommended Implementation Order..... 9

1. Overview & Design Principle

The Notification Service is split into 5 independent workstreams so each developer can raise exactly one Pull Request with no merge conflicts. It supports Email, SMS (mock), and In-App notifications, plus reusable Notification Templates and a Notification History query API, built on a layered architecture (Controller → Service → Repository → DTO → Entity) following SOLID principles.

Design principle that makes 5 independent PRs possible

Developer 1 owns the shared contract — the Notification entity (with every column any channel will ever need), the NotificationSender strategy interface, DTOs, exception hierarchy, and Swagger config. Developers 2–5 each add a new, self-contained vertical slice (own controller, own service implementation, own sender implementation) that implements Dev 1's interfaces but never edits Dev 1's files. No two developers touch the same file, so there are no merge conflicts regardless of PR order — only one sequencing rule: Dev 1 merges first.

Item	Value
Language / Runtime	Java 17
Framework	Spring Boot 3.x
Build Tool	Maven
Database	PostgreSQL
Persistence	Spring Data JPA
Validation	Jakarta Validation (Bean Validation)
API Documentation	springdoc-openapi (Swagger UI)
Testing	JUnit 5 + Mockito
Architecture	Layered: Controller → Service → Repository → DTO → Entity
Branching Strategy	Git feature-branch workflow, one PR per developer

2. Developer 1 — Core Foundation

Field	Detail
Task	Core Foundation — domain model, persistence, exception handling, and API documentation scaffolding that everyone else builds on.
Branch	feature/notification-core-foundation
APIs	POST /api/v1/notifications (generic create, reference implementation) GET /api/v1/notifications/{id} GET /api/v1/notifications?recipient=&status=&channel=&page=&size=
Classes	entity/Notification.java entity/NotificationTemplate.java (skeleton only — Dev 5 owns its body) enums/NotificationChannel.java (EMAIL, SMS, IN_APP) enums/NotificationStatus.java (PENDING, SENT, FAILED) repository/NotificationRepository.java dto/request/NotificationRequestDto.java dto/response/NotificationResponseDto.java dto/response/ApiErrorResponse.java service/NotificationSender.java (Strategy interface) service/NotificationService.java + impl/NotificationServiceImpl.java controller/NotificationController.java exception/NotificationServiceException.java exception/NotificationNotFoundException.java exception/GlobalExceptionHandler.java (@RestControllerAdvice) config/OpenApiConfig.java resources/db/migration/V1__create_notifications_table.sql resources/application.yml (PostgreSQL datasource)
Database	notifications table — id (PK, UUID), recipient, channel, subject, message, template_id (FK, nullable), status, retry_count, is_read, created_at, updated_at. This schema is designed up front to include every column Devs 2–5 need (is_read for in-app, template_id for templates) so nobody has to modify the entity later.
Acceptance Criteria	Entity, repository, and DTOs compile and are consumable by other modules; POST/GET endpoints return correct status codes (201/200/404); all exceptions produce a structured ApiErrorResponse via the global handler; Swagger UI loads at /swagger-ui.html and lists these endpoints; unit tests cover NotificationServiceImpl create/get paths with Mockito.
Commit Message	feat(notification): add core entity, repository, DTOs, exception handling, and Swagger config
PR Title	feat: Notification Service core foundation (entity, repository, exception handling, Swagger)

3. Developer 2 — Email Notification

Field	Detail
Task	Email Notification channel — real implementation behind the shared sender interface.
Branch	feature/notification-email
APIs	POST /api/v1/notifications/email
Classes	service/impl/EmailNotificationSender.java (implements NotificationSender) controller/EmailNotificationController.java dto/request/EmailNotificationRequestDto.java (@Email on recipient, @NotBlank subject/message) validation/EmailNotificationValidator.java (if a custom cross-field rule is needed) test/EmailNotificationSenderTest.java test/EmailNotificationControllerTest.java
Database	notifications table — writes rows with channel = 'EMAIL'. No schema change.
Acceptance Criteria	Valid request persists a PENDING row then transitions to SENT/FAILED; invalid email format returns 400 with a field-level message; endpoint documented in Swagger; ≥80% unit test coverage on the sender and controller; no modification to any file outside this slice.
Commit Message	feat(notification): implement email notification channel
PR Title	feat: Email notification channel (EmailNotificationSender + API)

4. Developer 3 — SMS Notification (Mock)

Field	Detail
Task	SMS Notification channel — mocked provider implementation.
Branch	feature/notification-sms
APIs	POST /api/v1/notifications/sms
Classes	service/impl/SmsNotificationSender.java (implements NotificationSender; logs + simulates success/failure via a configurable notification.sms.mock.failure-rate property) controller/SmsNotificationController.java dto/request/SmsNotificationRequestDto.java (@Pattern(regexp = "^\\+?[0-9]{7,15}\$") on phone) test/SmsNotificationSenderTest.java test/SmsNotificationControllerTest.java
Database	notifications table — writes rows with channel = 'SMS'. No schema change.
Acceptance Criteria	Valid request persists and dispatches through the mock sender; invalid phone format returns 400; failure-rate property is honored and covered by a unit test asserting both outcomes; endpoint documented in Swagger; no shared files touched.
Commit Message	feat(notification): implement mock SMS notification channel
PR Title	feat: SMS notification channel (mock SmsNotificationSender + API)

5. Developer 4 — In-App Notification

Field	Detail
Task	In-App Notification channel — send, list, and mark-as-read.
Branch	feature/notification-inapp
APIs	POST /api/v1/notifications/in-app GET /api/v1/notifications/in-app/user/{recipient} PATCH /api/v1/notifications/in-app/{id}/read
Classes	service/impl/InAppNotificationSender.java (implements NotificationSender) controller/InAppNotificationController.java dto/request/InAppNotificationRequestDto.java dto/response/InAppNotificationResponseDto.java (includes isRead) test/InAppNotificationSenderTest.java test/InAppNotificationControllerTest.java
Database	notifications table — writes channel = 'IN_APP', reads/updates the pre-existing is_read column from Dev 1's schema. No schema change.
Acceptance Criteria	Send persists with is_read = false; list endpoint returns paginated in-app notifications for a recipient; PATCH /read flips is_read to true and returns 200; reading a non-existent id returns 404 via the shared exception handler; Swagger documented; unit tests cover all three endpoints.
Commit Message	feat(notification): implement in-app notification channel with read tracking
PR Title	feat: In-app notification channel (send, list, mark-as-read)

6. Developer 5 — Templates & History

Field	Detail
Task	Notification Templates (CRUD) and Notification History (search/reporting).
Branch	feature/notification-templates-history
APIs	POST /api/v1/templates GET /api/v1/templates/{id} PUT /api/v1/templates/{id} DELETE /api/v1/templates/{id} GET /api/v1/notifications/history?recipient=&channel=&status=&page=&size=
Classes	entity/NotificationTemplate.java (completes the skeleton: id, name, channel, subjectTemplate, bodyTemplate, createdAt, updatedAt) repository/NotificationTemplateRepository.java dto/request/NotificationTemplateRequestDto.java dto/response/NotificationTemplateResponseDto.java service/NotificationTemplateService.java + impl/NotificationTemplateServiceImpl.java controller/NotificationTemplateController.java controller/NotificationHistoryController.java (read-only, queries NotificationRepository from Dev 1 — no writes, so no conflict) test/NotificationTemplateServiceTest.java test/NotificationHistoryControllerTest.java
Database	notification_templates (new table, this PR's own migration V2__create_notification_templates_table.sql) + notifications (read-only for history queries).
Acceptance Criteria	Full CRUD on templates returns correct status codes (201/200/204/404); duplicate template name returns 409; history endpoint supports filtering and pagination and returns 200 with an empty page (not an error) when no rows match; Swagger documented; unit tests cover the template service and history query logic.
Commit Message	feat(notification): implement notification templates CRUD and history search
PR Title	feat: Notification templates (CRUD) and notification history search

7. Recommended Implementation Order

Conflict-free merge sequence:

- Day 0 (all 5, ~30 min): agree on the Notification entity's full column list, the NotificationSender interface signature, and DTO field names — this is the contract everyone codes against, so Devs 2–5 don't have to wait idle for Dev 1's PR to literally merge before starting.
- Developer 1 merges first. Nobody else's branch can be merged before this one — every other slice implements NotificationSender and depends on the entity/repository/exception classes it introduces.
- Developers 2, 3, 4 rebase onto the merged foundation and open PRs in any order (Email, SMS, In-App) — they touch entirely disjoint files (different controllers, different sender implementations), so merge order among these three is arbitrary and none will conflict with the others.
- Developer 5 merges last, since the history endpoint is most meaningful to demo once Email/SMS/In-App rows actually exist in the table — not a technical dependency, just the most sensible order for integration testing and the sprint demo.
- Final integration check: after all 5 PRs are merged, run the full test suite together once to confirm no two independently-written unit tests collide on shared mock data or port bindings.

— End of Document —